HEINRICH-HEINE-UNIVERSITÄT DÜSSELDORF

Bachelor Thesis

Implementierung und Evaluation der Reinforcement-Learning-Methoden Self-Play und Liga-Training in der Micro-RTS-Umgebung

Name!!!

Degree:	B.Sc. Computer Science
Supervisor:	Professor Dr. Paul Swoboda
Co-Supervisor:	Dr. Peter Arndt
Faculty:	Faculty of Mathematics and Natural Sciences

22.08.2025

Abstract

Diese Arbeit verbessert einen bestehenden Agenten für das Echtzeit-Strategiespiel (RTS) MicroRTS. RTS-Spiele gelten als anspruchsvolle Domäne für Deep Reinforcement Learning, da Agenten mit großen kombinatorischen Aktionsräumen, simultanen Entscheidungen sowie verzögerten und oftmals spärlichen Belohnungen umgehen müssen. Als Basismethode verwenden wir Proximal Policy Optimization (PPO). Bei vielen reinforcement learning (RL) training arten, wie bei reinem PPO kommt man an einen Punkt, an dem der Agent nicht gut nur mit PPO weiter trainiert werden kann, weil ausreichend starke Gegner, gegen die man trainieren kann fehlen oder, weil die vorhandenen Gegner sind nicht divers genug sind, um eine robuste Policy zu entwickeln. Eine naheliegende und einfache Gegenmaßnahme ist Self-Play (SP), das jedoch in der Praxis instabil sein kann und zu Überanpassung oder zyklischen Strategien führt, wodurch die Leistung gegen externe Gegner sinken kann.

League-Training (LT) stabilisiert SP, indem es eine Population aus Agenten unterschiedlicher Trainingsstände und Rollen aufbaut. Der trainierenden Agent wird gezielt gegen vielfältige Gegner antreten. Unteranderem auch gegen Main-Exploiter, die trainiert wurden, Schwachstellen der aktuellen Policy auszunutzen. Wir evaluieren die resultierenden Agenten in Matches gegen verschiedene Gegnertypen und vergleichen LT mit reinem SP sowie unserem Anfangs Agent. Der Anfangs Agent wurde mit einer Mischung aus behavior cloning (BC) und PPO Trainiert. Unsere Ergebnisse zeigen, dass LT Robustheit und Generalisierungsfähigkeit des Agenten gegenüber unterschiedlichen Gegnern erhöht.

Contents

1. Introduction

RL etablierte sich in den letzten Jahren als leistungsstarker Ansatz für die Entwicklung von Agenten in komplexen Umgebungen. Spielumgebungen wie RTS-Spiele stellen dabei eine besondere Herausforderung dar, mit großen Aktionsräumen, langfristiger Planung und simultanen Entscheidungen. Allerdings bieten sie klar definierte Zustandsräume, Belohnungsstrukturen und wiederholbare Experimente, was sie zu idealen und herausfordernden Testumgebungen für die Erforschung von RL-Methoden macht.

Micro-RTS ist eine vereinfachte Version von RTS-Spielen, die speziell für die Forschung entwickelt wurde [Huang et al., 2021]. Sie ist diskret und reduziert die visuelle und mechanische Komplexität von großen kommerziellen RTS-Spielen. Wärenddessen behält sie die Kernschwierigkeiten, wie Ressourcenmanagement, Einheitenproduktion, Kampfmechaniken und taktische Positionierung bei. Micro-RTS eignet sich deshalb für die systematische Untersuchung von Lernverfahren. Micro-RTS behält allerding auch Trainingsherausforderungen, wie hohe kombinatorische Aktionsräume, verzögerte Belohnungen, Multi-Agent-Interaktionen und es erfordert auch robuster Strategien gegen unterschiedliche Gegner. Beispielsweise muss sich der Agent früh entscheiden, ob er gegen den aktuellen Gegner aggresiv oder defensiv spielen will, was sich auf die gesamte Spielstrategie auswirkt.

Ein Zentrales Problem in RL ist die Generalisierung gegenüber unterschiedlichen Gegnern und Strategien: Agenten lernen oft Strategien, die stark auf den Trainingsgegner oder eine spezifische Spielsituation zugeschnitten sind. Das führt zu Überanpassung und eingeschränkter Robustheit. SP, Fictitious Self-Play (FSP) und LT sind Ansätze die dieses Problem adressieren. In SP trainiert der Agent, in dem er gegen sich selbst spielt, was zu einem kontinuierlichen Lernprozess führt, der keine zusätzlichen diversen oder starken Gegner erfordert. Allerdings ist oft das Training instabil, weil es cyklisch lernt. Beispielsweise, wenn der Agentzustand B gut gegen A, C gut gegen B, und A gut gegen C ist, dann könnte der Agent in einem Zyklus gefangen sein, in dem er immer nur den Agentenzustand wieder erlernt, der gegen sich selbst gut ist, aber nicht gegen andere Gegner. Um die Instabilitäten von reinem SP zu adressieren, wird in FSP nicht gegen den aktuellen Agenten sondern gegen ältere Versionen des Agenten gespielt, was ein Cyklisches lernen verhindert und in zwei Spieler Nullsummenspielen gegen das Nash-gleichgewicht konvergiert. [Heinrich et al., 2015]. LT erweitert FSP, indem der Agent gegen historische Gegner aus einer Liga trainiert, die aus verschiedenen Agententypen besteht, beispielsweise aus historischen Trainingsständen oder spezialisierten Strategien [Vinyals et al., 2019]. Ziel ist es, die Lernumgebung zu stabelisieren, in der Agenten gegen ein breites Spektrum an Strategien antreten und dadurch robuster werden.

Trotz der theoretischen Vorteile ist die praktische Implementierung von LT in komplexen Umgebungen wie Micro-RTS mit erheblichen Herausforderungen verbunden. Dazu gehören:

- die effiziente Verwaltung und Auswahl von Gegnern für das Training des Main-Agenten und für die spezialisierten Agenten, wie Main-Exploiter
- die Stabilisierung des Trainingsprozesses bei nicht-konstanten Gegnerverteilungen
- der Umgang mit großen Aktionsräumen und spärlichen oder verzögerten Belohnungen
- sowie die Bewertung der tatsächlichen Generalisierungsfähigkeit der trainierten Agenten.

Darüberhinaus werden wir die für die unterschiedlichen Trainingsparadigmen die Lernkurve, Strategiediversität und Robustheit vergleichen und evaluieren.

Der Basis-Agent, den wir in diesem Projekt verwenden, wurde mit einer Kombination aus BC und PPO trainiert. BC ermöglicht es dem Agenten, schnell eine kompetente Basisstrategie zu erlernen, indem er Expertenverhalten imitiert. PPO ermöglicht es dem Agenten, durch Interaktion mit der Umgebung und Maximierung der erwarteten Belohnung über die anfängliche Kompetenz hinaus zu verbessern. Durch die Kombination von BC und PPO können wir die Vorteile beider Methoden nutzen: BC beschleunigt den Lernprozess, während PPO die Leistung durch Exploration weiter verbessert.

Modern video games offer researchers complex and diverse environments for developing and evaluating deep reinforcement learning (DRL) agents. Among these, real-time strategy (RTS) games have become a particularly important genre for AI research. They combine challenges such as large and variable action spaces, real-time decision-making constraints, and the need for concurrent strategic reasoning under continuous time pressure [Ontanón et al., 2013]. RTS environments, unlike turn-based games such as Chess or Go, require agents to manage multiple interdependent tasks simultaneously. These tasks include actions spanning over long stretches of time, like resource collection, as well as actions that require tactical planning and adaptation to the opponent's behavior, such as unit production and building construction. There are also challenges that demand high precision, where queuing an action just one timestep too late can determine the outcome of the entire match, such as in combat and unit movement. The complexity is further

increased by different unit types with distinct strengths and weaknesses, actions that can involve controlling multiple entities at once, and nondeterministic elements that may influence outcomes. Furthermore, partial observability limits the information available about the opponent's state, necessitating active scouting and exploration of the map, while the diversity of map layouts and sizes demands varied strategic adaptations. In addition, reward signals are often sparse and delayed until the end of a match, potentially thousands of timesteps after key decisions were made, making effective reward assignment particularly challenging. These characteristics collectively make RTS games a valuable and demanding testbed for evaluating approaches that integrate long-horizon planning, uncertainty handling, and real-time reactive control.

MicroRTS is a simplified yet still challenging RTS environment, designed specifically for AI research and benchmarking [Huang et al., 2021]. Unlike large-scale commercial titles such as StarCraft, MicroRTS offers a lightweight implementation with minimal engineering overhead, while retaining the core difficulties that make RTS games challenging for artificial intelligence. In particular, it features a large action space where agents must issue commands to multiple units simultaneously and partial observability through Fog of War. These properties require agents to integrate tactical reasoning while adapting in real time to an opponent's actions. Furthermore, the environment includes nondeterministic elements and diverse opponent strategies, which emphasize the need for robustness and generalization. This makes it an ideal platform for exploring new approaches to agent design without the heavy engineering and computational burden of a full-scale game.

In this work, two learning approaches are explored: behavior cloning (BC) and Proximal Policy Optimization (PPO)[1]. BC is a form of imitation learning where an agent learns to replicate expert behavior by training on recorded demonstrations, enabling it to quickly acquire a competent baseline policy [Torabi et al., 2018]. PPO is a DRL algorithm that improves an agent's policy by interacting with the environment and maximizing expected rewards, while maintaining stability through clipped policy updates [Schulman et al., 2017]. While BC can rapidly produce reasonable performance, it is limited by the quality and diversity of the expert data. PPO, on the other hand, can surpass expert performance through exploration, but it often struggles with sparse rewards and poor initial performance [Martin and Jhala, 2021]. Combining BC and PPO leverages the strengths of both methods using BC to bootstrap initial competence and PPO to refine the agent through exploration.

The goal of this thesis is to develop a MicroRTS agent that can consistently defeat the built-in scripted bots, which serve as a strong and diverse baseline. By combining BC with DRL, this hybrid pipeline not only accelerates training but also yields a competitive agent that achieves high win rates even against the strongest built-in opponents.

[1] https://github.com/mahuf100/MicroRTS_Maximilian_Huft

2. Related Work

MicroRTS-Py (Gym-MicroRTS) [Huang et al., 2021] introduced MicroRTS-Py, an OpenAI Gym wrapper for MicroRTS, which includes a PPO [Schulman et al., 2017] implementation that was trained on the `basesWorkers16x16A` map. In their work, they explored many techniques and different approaches to improve their agent. Their best performing approach incorporated action composition, a shaped reward function, invalid action masking, and the IMPALA-CNN [Espeholt et al., 2018] architecture with residual blocks [He et al., 2016]. Training was conducted against a diverse set of scripted agents, resulting in a 91% win rate on the target map against competition-level bots. Ablation studies revealed that invalid action masking was critical, its removal reduced the win rate to 0%. Additionally, in their paper "Comparing Observation and Action Representations for DRL in MicroRTS" they compared two action-generation strategies: Unit Action Simulation (UAS), which calls the policy sequentially for each unit and simulates intermediate states, and GridNet, which computes actions for all units in a single pass using per-grid-position logits and a player action mask [Huang and Ontanón, 2020b]. UAS achieved slightly higher performance (91% vs. 89%), but due to its complexity and limited compatibility with self-play and imitation learning, they decided to focus on GridNet and deprecated UAS.

They demonstrated that PPO with invalid-action masking and grid-wise action representations serves as a strong baseline for MicroRTS. Their work is widely used as a foundation in MicroRTS-based research and their gridnet_diverse_impala agent served as a baseline for this thesis.

RAISocketAI RAISocketAI won the IEEE-CoG 2023 MicroRTS competition, becoming the first DRL agent to claim the title in the competition's six-year history [Goodfriend, 2024]. It consistently outperformed previous champions, setting a new performance benchmark for DRL in MicroRTS. Its success was driven by iteratively fine-tuning the base policy and their training schedule. They also used different network architectures depending on the map size and applied transfer learning from agents trained on smaller maps to agents designed for larger maps to reduce training time. Additionally, after creating their competition winning agent, they also experimented with incorporating BC demonstrating its effectiveness as a

pretraining method for improving DRL initiation and the resulting boost to training speed.

Beyond the competition results, RAISocketAI contributed several enhancements to the MicroRTS-Py framework, including bug fixes and new functionality for recording bot-vs-bot replays. This replay recording capability allowed the creation of the demonstration dataset used for BC in this thesis.

BeClone BeClone demonstrated that combining BC with DRL fine-tuning can effectively improve agent performance in real-time strategy games, specifically in MicroRTS [Martin and Jhala, 2021]. Their approach follows a two-phase training process: first, BC is used with expert demonstration data to train a base policy, allowing the agent to achieve initial competence without relying on environment reward signals. Second, the policy is refined through using an Advantage Actor-Critic (A2C) algorithm, enabling dynamic strategy adaptation and more effective exploration. This method was used to create several bots for different map sizes with the goal of maximizing resource-gathering. The resulting agents outperformed the baseline agent from Huang and Ontañón [Huang et al., 2021] in both reward performance and training efficiency.

While experiments were restricted to resource-gathering tasks on small maps (maximum size 8×8), the results indicate that BC followed by DRL fine-tuning is a promising approach for MicroRTS and potentially other RTS domains.

AlphaStar [Vinyals et al., 2019] released AlphaStar, a grandmaster-level AI for StarCraft II trained using DRL. The system was initially bootstrapped with imitation learning sampling replays of the top-quartile of human players. The resulting supervised agents performed in the top 16% of human players and served as the starting point for DRL training. AlphaStar was trained using a self-play league in which a continuously evolving population of agents, including main competitors, exploiters, and past versions, trained against each other to enhance strategic robustness and prevent forgetting. Compared to StarCraft II, MicroRTS is far simpler in both complexity and simulation cost. The action and observation spaces also differ substantially between the two environments. In StarCraft's learning environment, AlphaStar's actions consist of: selecting an action type, selecting a subset of units and choosing a target. Once issued, these actions are executed until completion or interruption. In contrast, MicroRTS requires issuing discrete, single-step actions for each unit at every timestep. Despite the differences in scale, complexity, and computational resources available to DeepMind, AlphaStar served as a conceptual inspiration and informed several design and implementation choices in the development of the MicroRTS agent.

3. Method

3.1. Environment Setup

All experiments were conducted in the MicroRTS environment using RAISocketAI's reimplementation of MicroRTS-Py [Goodfriend, 2024]. All training and evaluations were performed on the `basesWorkers16x16A` map, commonly used in the MicroRTS research community [Huang et al., 2021, Ontanón et al., 2018]. The map contains a player base, an enemy base, a single initial worker for each player, and 4 resource patches, 2 per player. A screenshot of the game is shown in Figure 1. Green square tiles indicate resource nodes, with the overlaid numbers denoting remaining resources. Units outlined in blue belong to player 1, while units outlined in red belong to player 2. Light-gray square icons represent bases, and darker-gray squares denote barracks. Movable units are represented as circles. Small gray circles denote workers, which have low health and attack damage but can construct buildings and gather resources. Orange circles represent light units, which are fast moving with low health but relatively high attack damage. Blue circles indicate ranged units, which have low health and low attack damage but can attack enemies from up to three squares away. Yellow circles represent heavy units, which have high health and high attack damage. The unit types also differ in cost and production time. Workers cost only 1 and are produced quickly. Light and ranged units cost 2 and have medium production times. Heavy units cost 3 and require significantly longer to produce. The unit interactions follow a rock–paper–scissors relationship where: light units are effective against ranged units, ranged units are effective against heavy units, and heavy units are effective against light units [Richoux, 2020]. Partial observability (Fog of War) was disabled to simplify learning and provide the agent with the full game state at all times. The maximum episode length was set to 2,000 game steps. This limit was chosen because most matches on this map finish before the 2000-step mark, while those that do not typically take significantly longer to conclude.

The environment provides observations in the form of a channel-based spatial representation. The observation tensor has the shape (T, N, H, W, C), where:

- T — number of timesteps in the sequence
- N — number of parallel environments
- H — map height
- W — map width (usually the same as map height)

- C — number of feature channels (73 in this environment)

Each feature channel encodes a specific aspect of the game state, such as unit type, hit points, ownership, resource amounts, and production status. Since partial observability is disabled, these observations represent the complete game state at every step. In practice, the CNN backbone processes tensors of shape $(B, 73, 16, 16)$, where the batch dimension is defined as $B = T \times N$, flattening the temporal and environment axes.

The action space follows a multi-discrete per-tile encoding. Actions are represented as a tensor of shape $(T, N, H \times W, A)$, where $A = 7$ is the number of discrete action components. The seven independent discrete subactions are: action type, move direction, harvest direction, return direction, produce direction, produce type and relative attack position. For each tile on the map, the agent specifies a 7-dimensional action vector covering these subactions. Correspondingly, the actor head outputs logits of shape $(16 \times 16 \times 78)$, where the 78 dimensions arise from the expanded parameterization of the seven subactions. For example, the subaction "move direction" requires four discrete logits (north, south, east, west), while "produce type" requires one logit per unit type. Together, the concatenation of all subaction options yields 78 logits per tile, ensuring consistency between the per-tile multi-discrete encoding and the network's output.

To improve training efficiency, invalid action masking was applied to restrict the policy's choices to only valid actions for the current state. Invalid action masking is applied by setting the corresponding logits to a very large negative value, effectively assigning zero probability and gradient to actions that are illegal or have no effect (e.g., moving a unit that does not exist). This masking substantially reduces the effective action space per turn, improving training efficiency [Huang and Ontanón, 2020a].

To incorporate information about the source of demonstrations, during BC, the agent converts the expert's name into a learned embedding z, while simultaneously encoding the current observation using an observation encoder to produce its own estimate of the expert embedding. These two representations are combined using a blending factor α. Early in training, $\alpha \approx 1$, so the agent relies primarily on the expert-provided z. Over 30 epochs, α linearly decreases to zero, gradually shifting the agent's reliance toward the observation-based encoding. This allows the encoder to learn associations between gameplay patterns and expert identity. In subsequent PPO training, the observation encoder alone determines z, ensuring the network can both leverage multiple expert styles and adapt to new situations.

In addition, a scalar feature vector is extracted from the observation to capture global game statistics, similar to the scalar features used in [Vinyals et al., 2019]. This scalar vector contains 11 features:

- Player 0 stored resources
- Player 1 stored resources
- Available resources left on the map
- Player 0: number of workers, light units, heavy units, ranged units
- Player 1: number of workers, light units, heavy units, ranged units

These features are processed by a dedicated encoder before being concatenated with z and the CNN backbone features.

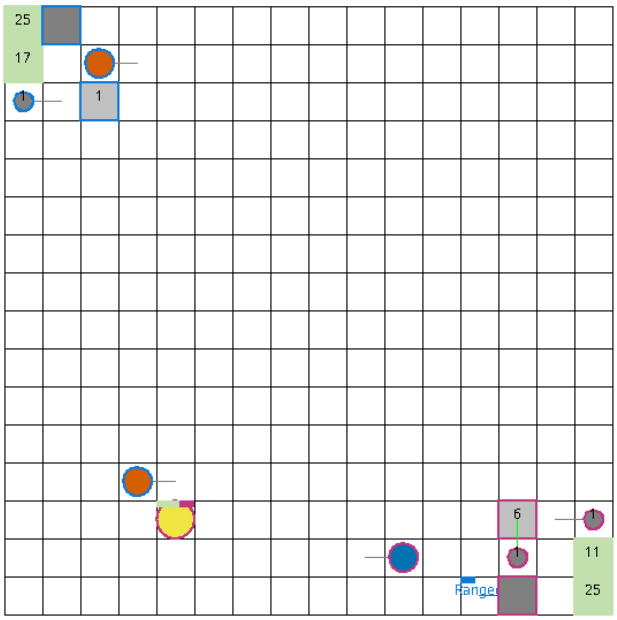

Figure 1. Screenshot of the MicroRTS `basesWorkers16x16A` map, including all units and buildings.

3.2. Neural Network Architecture

The agent's policy and value networks share a common convolutional backbone and separate linear heads for actor and critic. The backbone (Figure 2) processes the input observation tensor of shape $(B, 73, 16, 16)$ and produces a 256-dimensional feature representation for each batch element, resulting in an output tensor of shape $(B, 256)$.

It uses residual connections implemented as residual blocks (ResBlocks) [He et al., 2016]. The ResBlocks apply two 3×3 convolutions with padding 1: the first convolution is followed by a GELU activation, while the second is followed by a Squeeze-and-Excitation (SE) block. The SE block compresses each channel via global average

pooling. Then the pooled vector goes through a bottleneck 1×1 convolution that reduces channel dimensionality by a factor of 16, followed by a GELU activation. After the original channel dimensionality is restored with a second 1×1 convolution and a sigmoid activation to produce channel-wise weights. The output of the second convolution is reweighted by these channel weights and added to the original input via a residual connection and a final GELU activation is applied to the sum. This design enables the block to extract local spatial features, dynamically emphasize informative channels, and improve gradient flow for stable training of deeper networks.

Two auxiliary encoding modules are used alongside the convolutional backbone: a scalar feature encoder that compresses global, non-spatial game statistics into a fixed-size vector, and a zEncoder that produces a compact latent conditioning vector from the observations.

ScalarEncoder The ScalarEncoder is a small multi-layer perceptron that maps a compact set of global game statistics to a feature vector used by both actor and critic. It consists of $L = 2$ fully connected layers. The first layer projects the input of dimension d_{in} to a hidden layer of width 32 with ReLU activation, and the second maps this hidden vector to an output space of dimension 32:

$$\text{ScalarEncoder}: \ \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}},$$

In this implementation $d_{\text{in}} = 11$ and $d_{\text{out}} = 32$. This dense representation summarizes global information such as resources and unit counts into a compact form that complements spatial features extracted by the convolutional backbone. It provides both actor and critic networks with access to global context, improving strategic decision-making beyond purely spatial information.

zEncoder. The zEncoder is a lightweight feed-forward network that maps the current observation into a compact latent code z. It is implemented as a two-layer perceptron with an intermediate hidden size of 64 and a ReLU activation in between them:

$$\text{zEncoder}: \ \mathbb{R}^{d_{\text{obs}}} \to \mathbb{R}^{d_z},$$

In this work the flattened observation dimension $d_{\text{obs}} = 16 \times 16 \times 73$, and the latent dimension $d_{\text{z}} = 8$. This structure provides a dense, low-dimensional summary of the observation. During the BC phase, the output is combined with an expert-provided embedding via a blending schedule, allowing the zEncoder to learn associations between gameplay patterns and expert identity. This enables the policy to exploit diverse expert behaviors during BC while retaining the flexibility to generalize. The ScalarEncoder supplies explicit, low-dimensional global statistics that support value estimation and high-level decision making. The zEncoder produces a dense latent conditioning vector that, together with a compact expert identity embedding, enables the policy to represent broader contextual factors. This design allows the agent to condition on demonstration style during BC, thereby capturing different expert playstyles, and later to adapt to varied contexts during PPO fine-tuning.

The ScalarEncoder and zEncoder are concatenated with the CNN-produced spatial feature vector to form the final state representation fed to the actor and critic heads:

$$h = \left[h_{\text{cnn}} \,\|\, h_{\text{scalar}} \,\|\, h_z\right] \in \mathbb{R}^{256+32+8}.$$

Two separate linear heads, map the concatenated representation h to policy logits and a scalar state value:

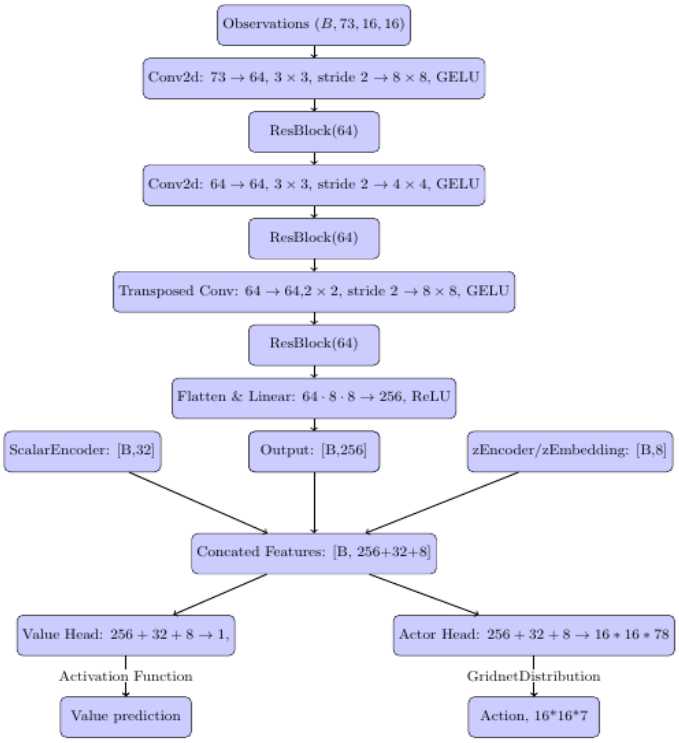

Figure 2. The Network Architecture

- actor$(h) = \text{Linear}\big(256 + 32 + 8 \to M\big)$ — produces logits for the multi-discrete per-tile action encoding. In this implementation $M =$ mapsize $\times$ action_plane_size $= 16 \times 16 \times 78$, i.e. a block of logits per map tile covering all action components.
- critic$(h) = \text{Linear}\big(256 + 32 + 8 \to 1\big)$ — predicts the state value.

The layer weights are initialized using orthogonal initialization with a configurable standard deviation, while biases are set to a constant value [Schulman et al., 2017]. The actor head uses a small standard deviation (0.01) to keep initial logits near zero, while the critic uses a larger standard deviation(1.0).

The architecture incorporates several key design principles that facilitate effective policy learning in the RTS domain. First, GELU activations are used throughout the convolutional pathway and within the ResBlocks, while a ReLU activation is applied after the final linear projection of the CNN backbone before concatenation with auxiliary encoders. This combination improves non-linear expressivity while maintaining numerical stability. Residual connections further alleviate vanishing gradients and enable deeper feature processing [He et al., 2016], while SE attention adaptively re-weights channel activations, allowing the network to focus on informative features across different game contexts [Hu et al., 2018].

To balance efficiency and spatial reasoning, the model employs stride-2 convolutions for downsampling followed by transpose convolutions for upsampling. This reduces computational cost while preserving sufficient spatial context for per-tile action decoding. Beyond spatial features, auxiliary encoders process scalar statistics and a latent variable z, which together supply global, non-spatial, and contextual information that cannot be captured by convolutional processing alone.

Finally, the policy head implements a per-tile multi-discrete action representation, directly aligned with the environment's action encoding. This design enables the network to output actions for all map locations in parallel, ensuring efficient and scalable policy execution [Huang and Ontanón, 2020b].

3.3. Behavior Cloning

For the BC phase, expert demonstrations were generated by having the 2020 competition winner CoacAI and the 2021 competition winner Mayari play against all built-in bots in MicroRTS. The bot environment was configured to be nondeterministic, introducing small variations in the replays and thus producing more diverse training data and the maximum amount of timesteps per game was set to 2000. The same number of replays from CoacAI and Mayari was used, with 50 to 300 replays collected per matchup. Harder and more random opponents were prioritized over easier ones, which tend to produce repetitive game states, resulting in a total of 5400 replays comprising approximately 7 million game states. Additionally, a secondary dataset was collected, filtering for only the games in which the expert won. This resulted in a smaller dataset, containing roughly 4100 games with around 5.2 million game states. This dataset was used for fine-tuning the agent following the initial BC training phase.

The expert observations are fed into the policy network, and the resulting logits are compared to the expert actions to compute the BC loss. The objective function is defined as:

$$\mathcal{L}_{\text{BC}}(\theta) = -\frac{1}{B \cdot H \cdot W} \sum_{b,h,w} \log \pi_\theta(a_{b,h,w}^{\text{expert}} \mid s_{b,h,w}),$$

Here B denotes the batch size, (H, W) the map height and width, s the observation, a^{expert} the expert actions, and π_θ the policy with parameters θ. The summation runs over all batch elements b and spatial locations (h, w), i.e., every tile in the input maps. The loss is normalized by the total number of tiles in the batch $(B \cdot H \cdot W)$. This per-tile normalization ensures a consistent scaling of the loss across different batch and map sizes

3.4. Reinforcement Training

After the BC phase, the supervised policy was further fine-tuned using DRL to improve its performance and adaptability beyond the scope of the expert demonstrations. The PPO algorithm was employed for this stage. The same environment configurations were used to ensure continuity between the BC and RL phases. Two rewards were used during training, a win-loss reward and an attack reward. The win-loss reward is 10 for a win, -10 for a loss and 0 for a draw. This reward is further scaled by a linear function, which is approximately 1.1 at 500 steps and decreases to about 0.9 at 2000 steps. This encourages agents to win games quickly and to prolong games when losing, thereby rewarding faster victories and slower defeats. After BC training, the agent continued to exhibit difficulties in consistently attacking nearby enemy units, either delaying the attack or not attacking at all. To address this, an attack reward was introduced, granting the agent a reward for each valid attack. This attack reward is much smaller than the win-loss reward, set at 0.05 per attack and averages approximately 0.9 reward per game. This reward helps the agent in learning to attack an enemy unit, similar to the attack reward used in [Goodfriend, 2024], while still remaining small enough not to interfere with the primary objective of winning the game. Only these two rewards were used, since the agent should already have learned basic MicroRTS mechanics such as resource gathering, unit training, and unit control.

During reinforcement training, the agent was trained against a mix of built-in MicroRTS bots. This training setup encouraged the agent to adapt dynamically to different opponent strategies while exploring parts of the action space [Huang et al., 2021].

The policy is trained using the PPO loss

function [Schulman et al., 2017]:

$$L(\theta) = \hat{\mathbb{E}}_t\Big[L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 S\big[\pi_\theta\big](s_t)\Big],$$

where the clipped surrogate objective is

$$L^{\text{CLIP}}(\theta) = \min\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}_t,\ \text{clip}\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right).$$

and the value function loss is

$$L^{\text{VF}}(\theta) = \tfrac{1}{2}\left(V_\theta(s_t) - \hat{V}_t\right)^2.$$

where c_1 is the value loss coefficient, c_2 is the entropy coefficient, S is the entropy bonus function, π_θ is the stochastic policy, $\pi_{\theta_{\text{old}}}$ is the rollout policy, a_t is the action taken at time t, s_t is the observation at time t, $\hat{A}_t$ is the advantage estimate, ϵ is the clipping coefficient, V_θ is the value function, and $\hat{V}_t$ is the return estimate. The advantage is estimated using Generalized Advantage Estimation (GAE) [Schulman et al., 2015], with hyperparameters γ (discount factor) and λ (exponential weighting). GAE is employed to reduce the variance of the advantage estimates while introducing only a small bias, which stabilizes policy updates and improves learning efficiency. The resulting advantage estimates are then used in the clipped policy loss $\mathcal{L}_{\text{CLIP}}$.

To prevent the policy from deviating too far from the supervised BC policy, we include a KL divergence regularization term. For each minibatch, the KL divergence is computed between the current policy π_θ and the target supervised policy π_{target} over all per-tile discrete action distributions:

$$\text{KL}_{\text{div}} = \frac{1}{B}\sum_{b=1}^{B} \text{KL}\left(\pi_{\text{BC}}^{(b)} \,\|\, \pi_\theta^{(b)}\right),$$

where B is the minibatch size. Both π_θ and π_{BC} are obtained by applying the softmax function to the respective logits. The KL divergence is then scaled by a coefficient λ_{KL} and then added to the overall loss. This regularization encourages the learned policy to remain close to the BC policy, stabilizing learning and preventing overconfident deviations during PPO training.

4. Experiments

4.1. Setup

The agent was trained in three sequential stages: BC, BC fine-tuning (BC-FT) and PPO.

The initial BC phase used a total of 5400 replays, generated against the built-in Mayari and CoacAI opponents (see Section 3.3 for replay generation details). Training ran for 100 epochs with the Adam optimizer, an initial learning rate of 1×10^{-3}, and a batch size of 2048. The BC loss plateaued after 100 epochs, with no further improvement. Afterwards, the agent was fine-tuned using another dataset of roughly 4100 replays that only included games where the expert won, with identical optimizer settings and batch size. The model converged within 70 epochs, only requiring roughly half the training time needed during the initial BC phase. During both phases, linear learning rate annealing was used, which proved particularly beneficial, enabling a higher initial learning rate that accelerated early learning while stabilizing convergence later on.

Table 1. Training parameters for PPO training.

Parameter	**Value**
Initial learning rate (Adam)	2.5×10^{-5}
Rollout steps per environment	512
Minibatch Size	3072
Max episode length	2,000 timesteps
Discount factor γ	1
Value function coefficient β	0.01
Entropy coefficient η	0.005
Gradient norm threshold ω	0.5
KL regularization coefficient λ_{KL}	0.4
GAE parameter λ	0.95

The BC-FT policy was further optimized using PPO with 24 parallel game environments. The best performance was achieved when only training against CoacAI and Mayari. 16 CoacAI and 8 Mayari environments were used, based on observations that the agent performed better against Mayari during training. PPO used a rollout size of 512 leading to a total rollout size of 24 $\times$ 512 = 12288 timesteps. Using a larger rollout size compared to [Huang et al., 2021] proved vital, as rewards in this work were much sparser, focusing mainly on the win-loss reward. Four mini-batches and four PPO updates were used per rollout, resulting in a minibatch size of 3072 and a total of 16 gradient steps per rollout, which allowed the agent to efficiently process experience while stabilizing training. PPO was configured with an initial learning rate of 2.5×10^{-5}, $\lambda_{\text{GAE}} = 0.95$, entropy coefficient 0.005, value coefficient 0.01, maximum gradient norm 0.5, clipping coefficient 0.1 and $\gamma = 1.0$ (no discount applied) in order to accurately reflect the true objective of winning games. The learning rate was linearly annealed throughout training to improve convergence and stabilize learning. The parameter values were chosen based on prior work [Goodfriend, 2024, Vinyals et al., 2019] and through empirical experimentation with different settings. Training

was run for 30 million steps, after which both episode reward and win rate plateaued.

At the end of each stage (BC, BC-FT, PPO), a total of 100 deterministic evaluation games were conducted against all built-in bots in MicroRTS. All evaluations were conducted on the `basesWorkers16x16A` map with both partial observability and nondeterminism disabled.

Table 2. Win rates (%) of BC, BC-FT, and after PPO against each bot.

Bot	BC	BC-FT	PPO
WorkerRushAI	3	50	99
PassiveAI	98	100	100
LightRushAI	22	93	99
CoacAI	0	8	96
Mayari	0	5	95
RandomAI	52	95	99
RandomBiasedAI	18	68	86
rojo	0	34	84
mixedBot	4	44	59
izanagi	0	62	61
tiamat	11	78	84
droplet	0	31	59
guidedRojoA3N	4	42	75
naiveMCTS	0	5	3

4.2. Results

We evaluated our agent across three stages: behavior cloning (BC), fine-tuned behavior cloning (BC-FT), and reinforcement learning with Proximal Policy Optimization (PPO). Table 2 summarizes overall win rates, showing clear performance improvements across stages.

The initial BC phase produced policies capable of imitating basic expert behaviors, such as resource gathering and unit production, but the agent still struggled with unit control and combat actions. While the agent occasionally defeated weaker bots, it could not consistently secure victories and failed against stronger opponents.

Fine-tuning on a smaller dataset containing only winning replays led to noticeable improvements. Leveraging the pretrained policy with this focused dataset reduced training time and allowed the agent to reliably defeat weaker bots, though it still rarely overcame stronger adversaries. The resulting agent was able to perform more advanced tasks, such as unit control and combat, but frequent small mistakes and inconsistencies often led to losses against stronger opponents, confirming that imitation learning alone is insufficient for mastering MicroRTS. The overall win rate improved from 20.3% (BC) to 54.8% (BC-FT).

The largest gains came from PPO training. After fewer than 30 million steps, the agent achieved near-perfect win rates against weaker bots and competitive results against strong opponents such as CoacAI and Mayari. Targeted training on these two bots not only boosted direct performance but also generalized well to other strong bots. Expanding the opponent pool during PPO training decreased performance, emphasizing the importance of opponent selection. During PPO training the overall win rate improved to 88.6%.

The final agent achieved similar or higher win rates than the gridnet_diverse_impala agent [Huang et al., 2021] against most bots. For example, against LightRushAI, the final agent achieved a win rate of 99%, compared to 79% for the baseline, and against Izanagi, the final agent achieved 61% compared to 48%. Some exceptions were RandomBiasedAI (86% vs. 100%), NaiveMCTS (3% vs. 14%), rojo (84% vs. 100%) and guidedRojoA3N (75% vs. 91%). Against CoacAI, the final agent achieved a 96% win rate, even managing to outperform the RAISocketAI agent's 90% win rate on the same map with identical settings [Goodfriend, 2024]. These results were obtained using only 30 million timesteps, compared to the 300 million timesteps required by both the gridnet_diverse_impala agent and RAISocketAI. Despite achieving a high overall win rate, the final agent struggled against highly stochastic bots (Rojo, RandomBiasedAI, NaiveMCTS) where almost all non-wins were draws caused by single units hiding in corners of the map. Attempting to improve performance against these bots by training exclusively against RandomBiasedAI proved ineffective, reducing both the win rate and subsequently episode reward, as shown in Figure 4. The agent also demonstrated difficulties in controlling ranged units effectively. Instead of exploiting their longer attack distance, it often moved them into melee range, thereby negating their advantage. As a consequence, the policy tended to favor producing heavy units, which are more robust in close combat. Light units were not used at all, due to the expert demonstrations from CoacAI and Mayari exclusively using heavy and ranged units.

Figure 3 shows average win rates during PPO training under different opponent configurations, where training with 16 CoacAI and 8 Mayari yielded the strongest results. Extending training beyond 30 million steps yielded no improvement, as seen in Figure 3. Figure 4 illustrates that training the final agent on a broader set of opponents also did not seem to have any significant effect on the win rate, indicating a trade-off between opponent diversity and specialization.

Overall, our results demonstrate that combining BC, targeted fine-tuning, and PPO yields substantial improvements, especially inregards to training time, with opponent selection and learning rate scheduling being critical factors in achieving high performance.

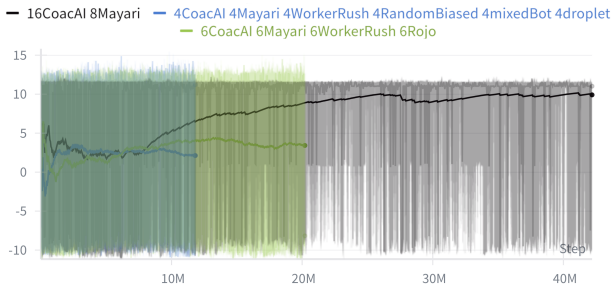

Figure 3. Episode rewards (y-axis) as a function of training timesteps (x-axis) on the BC-FT agent.

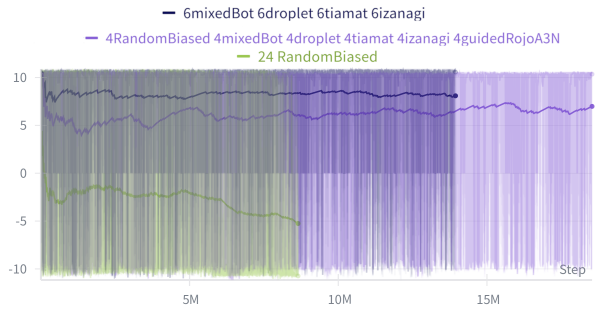

Figure 4. Episode rewards (y-axis) as a function of training timesteps (x-axis) on the final agent.

4.3. Discussion

Together, these results highlight the complementary strengths of BC and DRL. BC bootstrapped a competent policy, bypassing the need to learn basic mechanics such as resource gathering and unit production. Fine-tuning the policy on winning replays accelerated the acquisition of higher-level skills, such as coordinated unit control, while also reinforcing the previously learned fundamentals. PPO then refined these policies through exploration and adaptation, achieving near-perfect results against weaker bots and competitive performance against stronger adversaries such as CoacAI and Mayari.

Training exclusively against CoacAI and Mayari not only improved performance against them, but also generalized well to other strong adversaries. Experiments with broader or alternative opponent pools often yielded no improvement and, in some cases, degraded performance. This tension between opponent diversity and specialization reflects a classic bias–variance trade-off where training on a narrow pool risks overfitting to specific strategies, while expanding the opponent set can introduce variance that hinders learning progress. Balancing these competing factors remains a key challenge for scaling to more robust and general policies.

Overall, this hybrid approach demonstrates that BC can rapidly bootstrap policies in complex action spaces, while PPO fine-tunes strategies and adapts to challenging scenarios, highlighting the benefits of targeted opponent selection.

Despite the overall success, several limitations were identified:

- **Generalization to random opponents:** The agent struggled with highly random bots such as RandomBiasedAI and NaiveMCTS, where most nonwins were draws caused by a single unit hiding in a corner of the map. This demonstrates that the agent has issues when encountering new or unseen game states.
- **Map and observability constraints:** All experiments were conducted on the `basesWorkers16x16A` map without partial observability. Scaling to larger maps or Fog of War would require broader and more diverse replay datasets.
- **Replay and training bottlenecks:** Collecting replays and training against compute-heavy bots (e.g.,guidedRojoA3N, NaiveMCTS, Droplet) significantly slowed the process, making it inefficient to train on them.
- **Exploitability:** The agent was trained exclusively using built-in bots and demonstrated limited exposure to new and unexplored game states. As a result, it remains vulnerable to opponents employing unconventional strategies, such as attacking from inefficient routes or deliberately hiding units when losing, which could be easily exploited.

Several avenues could extend and improve the current work. Self-play was not utilized in this study to avoid overcomplicating training. However, several studies have demonstrated the benefits of self-play in MicroRTS [Goodfriend, 2024], [Huang et al., 2021]. Implementing a self-play league system, where different versions of the agent compete against each other and are grouped into distinct agent types, like the approach used in AlphaStar [Vinyals et al., 2019], appears particularly promising for further performance improvements. Scaling to different maps and exploring different game settings like partial observability turned on need to be addressed before the agent can be considered fully functional. [Goodfriend, 2024] showed the potential of using transfer learning to different maps, as well as using different network architectures depending on the size of the map. Expanding the replay dataset with demonstrations that include game states not present in the current dataset, such as starting the expert agent from a disadvantaged position or from

positions it would not normally encounter, could improve generalization and robustness. Similarly, incorporating replays from stronger and more diverse opponents, such as other competition winners and participants, could further enhance performance. Investigating persistent points of failure, such as enemy units hiding in corners, by developing strategies or reward shaping to address rare or unexplored game states could improve the agent's ability to handle such situations. It is also important to note that during the IEEE-CoG MicroRTS competition, agents are limited to 100 ms per timestep. While the current agent does not exceed this limit, future implementations, especially on larger and more complex maps, may require more computation time and risk surpassing the time budget.

5. Conclusion

This thesis presented the development of a competitive MicroRTS agent trained using a combination of behavior cloning and Proximal Policy Optimization. Starting from expert demonstrations, the agent learned to imitate strong play and was subsequently fine-tuned through deep reinforcement learning to adapt and improve in dynamic game situations. The results indicate that while behavior cloning accelerates early-stage training, reinforcement learning remains essential for creating a competitive agent in MicroRTS.

The resulting agent outperformed or achieved comparable performance to existing baselines on most opponents, attaining consistently strong win rates, even against some of the strongest bots, while requiring only a fraction of the training time. Training primarily on winning replays accelerated the acquisition of advanced skills such as unit coordination, tactical combat, and broader strategic decision-making, while the careful selection of challenging opponents promoted generalization to unseen matchups. These results demonstrate that behavior cloning, when combined with PPO fine-tuning, can effectively tackle the large and complex action spaces characteristic of real-time strategy games.

Beyond MicroRTS, this study reinforces the viability of combining imitation learning and reinforcement learning in domains with large action spaces and sparse rewards. These findings may extend to other RTS benchmarks like StarCraft II or to broader sequential decision-making problems with similar characteristics.

Despite these successes, the study was limited to the `basesWorkers16x16A` map with Fog of War disabled, and opponent diversity during training and evaluation was constrained. Consequently, the agent's adaptability to larger maps, partially observable settings, or entirely new opponents remains untested. Future work should focus on enabling partial observability, scaling to larger and more diverse maps, increasing opponent diversity, and leveraging advanced demonstration data. Implementing self-play and transfer learning may further enhance robustness and adaptability, while careful attention to computational constraints will ensure feasibility in competition settings. Addressing these areas is essential for developing future agents that are fully competition-ready in MicroRTS. In summary, this work demonstrates that combining imitation learning with PPO fine-tuning is a powerful paradigm for tackling the challenges of RTS games. By addressing scalability, diversity, and uncertainty, future agents could not only become strong MicroRTS competitors but also contribute to broader progress in creating DRL agents that must plan, act, and learn in complex real-time environments with large action spaces.

References

[Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). Impala: scalable distributed deep-rl with importance weighted actor-learner architectures. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 1406–1415. 3

[Goodfriend, 2024] Goodfriend, S. (2024). Raisocketai: The first deep reinforcement learning agent to win the ieee microrts competition. In *2024 IEEE Conference on Games (CoG)*, pages 1–8. 3, 4, 7, 8, 9, 10

[He et al., 2016] He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*. 3, 5, 7

[Heinrich et al., 2015] Heinrich, J., Lanctot, M., and Silver, D. (2015). Fictitious self-play in extensive-form games. In Bach, F. and Blei, D., editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 805–813. 2

[Hu et al., 2018] Hu, J., Shen, L., Albanie, S., Sun, G., and Wu, E. (2018). Squeeze-and-excitation networks. *arXiv preprint arXiv:1709.01507*. 7

[Huang and Ontanón, 2020a] Huang, S. and Ontanón, S. (2020a). A closer look at invalid action masking in policy gradient algorithms. *CoRR*, abs/2006.14171. 5

[Huang and Ontanón, 2020b] Huang, S. and Ontanón, S. (2020b). Comparing observation and action representations. *arXiv preprint arXiv:1910.12134*. 3, 7

[Huang et al., 2021] Huang, S., Ontanón, S., Bamford, C., and Grela, L. (2021). Gym-microrts: Toward affordable full game real-time strategy games research with deep reinforcement learning. *CoRR*, abs/2105.13807. 2, 3, 4, 7, 8, 9, 10

[Martin and Jhala, 2021] Martin, D. and Jhala, A. (2021). Beclone: Behavior cloning with inference for real-time strategy games. In *Joint Proceedings of the AIIDE 2021 Workshops*. 3, 4

[Ontanón et al., 2018] Ontanón, S., Barriga, N. A., Silva, C. R., Moraes, R. O., and Lelis, L. H. S. (2018). The first microrts artificial intelligence competition. *AI Magazine*, 39:75–83. 4

[Ontanón et al., 2013] Ontanón, S., Synnaeve, G., Uriarte, A., Richoux, F., Churchill, D., and Preuss, M. (2013). A survey of real-time strategy game ai research and competition in starcraft. *IEEE Transactions on Computational Intelligence and AI in Games*, 5(4):293–311. 2

[Richoux, 2020] Richoux, F. (2020). microphantom: Playing microrts under uncertainty and chaos. *arXiv preprint arXiv:2005.11019*. 4

[Schulman et al., 2015] Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. (2015). High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*. 8

[Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *CoRR*, abs/1707.06347. 3, 7, 8

[Torabi et al., 2018] Torabi, F., Warnell, G., and Stone, P. (2018). Behavioral cloning from observation. *arXiv preprint arXiv:1805.01954*. 3

[Vinyals et al., 2019] Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., Vezhnevets, A. S., Leblond, R., Pohlen, T., Dalibard, V., Budden, D., Sulsky, Y., Molloy, J., Paine, T. L., Gulcehre, C., Wang, Z., Pfaff, T., Wu, Y., Ring, R., Yogatama, D., Wünsch, D., McKinney, K., Smith, O., Schaul, T., Lillicrap, T., Kavukcuoglu, K., Hassabis, D., Apps, C., and Silver, D. (2019). Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354. 2, 4, 5, 8, 10

Erklärung

Hiermit versichere ich, dass ich die Bachelorarbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Düsseldorf, den 22.08.2025

Niklas Glaser